A Rigorous Proof Framework for Lower Bounds on the Autoconvolution Constant C_{1a}

Via the Cloninger–Steinerberger Branch-and-Prune Algorithm

Jai Bajaj

March 19, 2026

Abstract

We present a self-contained proof that the autoconvolution constant C_{1a} — the infimum of $\|f * f\|_{L^\infty}/\left(\int f\right)^2$ over nonnegative functions supported on $[-\frac{1}{4}, \frac{1}{4}]$ — satisfies a lower bound certifiable by exhaustive computation. Our starting point is the framework of Cloninger and Steinerberger (2017), which we extend with a multiscale cascade and tighter pruning. The paper is organized around the natural sequence of reductions: from continuous functions to bin averages (Section 2), from the continuous simplex to a finite lattice (Section 3), and finally to an exhaustive search made tractable by pruning (Sections 4–6). Section 8 addresses the invariants that keep the computation honest. Throughout, we emphasize intuition alongside formal statements.

Contents

1 Introduction **3**
1.1 The Constant C_{1a} . . . 3
1.2 Known Bounds . . . 3
1.3 Proof Strategy at a High Level . . . 4

2 From Functions to Bin Averages **4**
2.1 The Partition . . . 4
2.2 Bin Averages . . . 4
2.3 The Lower Bound on $\|f * f\|_\infty$ . . . 5

3 Discretization of the Simplex **6**
3.1 The Lattice . . . 6
3.2 The Discretization Error Bound . . . 7
3.3 The Refined (Dynamic) Correction . . . 7

4 The Windowed Test Value **8**
4.1 Definition . . . 8
4.2 Efficient Computation via Prefix Sums . . . 9
4.3 Reversal Symmetry . . . 9

5 Pruning Strategies **10**
5.1 Asymmetry Pruning . . . 10
5.2 Canonical Symmetry Exploitation . . . 11

6 The Multiscale Cascade **12**
6.1 Overview . . . 12
6.2 Dyadic Splitting (Refinement) . . . 12
6.3 Energy Cap . . . 12
6.4 Cascade Coverage . . . 13
6.5 Canonical Handling Across Levels . . . 13
6.6 Fused Generate-and-Prune Kernel . . . 14

7 The Complete Proof **15**
7.1 Algorithm . . . 15
7.2 Main Theorem . . . 15

8 Correctness Invariants **16**
8.1 Invariant 1: Complete Enumeration . . . 16
8.2 Invariant 2: All Windows Checked . . . 16
8.3 Invariant 3: Conservative Arithmetic . . . 16
8.4 Invariant 4: Refinement Completeness . . . 17

9 Computational Verification **17**
9.1 Composition Generators . . . 17
9.2 Test Value Computation . . . 17
9.3 Pruning Correctness . . . 17
9.4 MATLAB Baseline Comparison . . . 18
9.5 Lean Formalization Status . . . 18

10 Discussion **18**
10.1 Relationship to the Original CS17 Proof . . . 18
10.2 Limitations . . . 18

1 Introduction

1.1 The Constant C_{1a}

Consider the class of nonnegative, integrable functions supported on the interval $[-\frac{1}{4}, \frac{1}{4}]$:

$$\mathcal{F} \;=\; \Big\{ f : \mathbb{R} \to \mathbb{R}_{\geq 0} \;\Big|\; \operatorname{supp}(f) \subseteq \big[-\tfrac{1}{4}, \tfrac{1}{4}\big],\; \int f > 0 \Big\}.$$

The *autoconvolution* of f is

$$(f * f)(t) \;=\; \int_{-\infty}^{\infty} f(x)\, f(t-x)\, dx.$$

Since $\operatorname{supp}(f) \subseteq [-\frac{1}{4}, \frac{1}{4}]$, the autoconvolution is supported on $[-\frac{1}{2}, \frac{1}{2}]$ and is itself nonnegative.

A natural question is: how small can the peak of $f * f$ be, relative to the total mass of f? This is precisely what C_{1a} measures.

Definition 1.1. The *autoconvolution constant* is

$$C_{1a} \;=\; \inf_{f \in \mathcal{F}} \frac{\max\limits_{|t| \leq 1/2} (f * f)(t)}{\left(\int f \right)^2}.$$

By homogeneity, we may normalize $\int f = 1$ without loss of generality, so that $C_{1a} = \inf_f \|f * f\|_{L^\infty}$ over nonnegative f with $\operatorname{supp}(f) \subseteq [-\frac{1}{4}, \frac{1}{4}]$ and $\int f = 1$.

Why this constant matters. C_{1a} is connected to the asymptotic size of Sidon sets in additive combinatorics. A *Sidon set* is a set of integers where all pairwise sums are distinct; equivalently, the "representation function" of the sumset has values at most 2 everywhere except at doubled elements. The constant C_{1a} governs how efficiently mass can be distributed to minimize the peak of the autoconvolution — the continuous analog of controlling pairwise-sum collisions.

1.2 Known Bounds

Bound	**Value**	**Reference**
Upper bound	≤ 1.5029	Matolcsi & Vinuesa (2010)
Lower bound	≥ 1.2802	Cloninger & Steinerberger (2017)

The *upper bound* comes from exhibiting explicit functions whose autoconvolution peak is small. This is a constructive task: find a good f and compute its peak.

The *lower bound* is fundamentally harder: one must show that *no* function can achieve a peak below the threshold. This requires exhaustive elimination of all possibilities, which is the subject of this document.

1.3 Proof Strategy at a High Level

The proof proceeds in three conceptual steps:

(i) **Reduce** the infinite-dimensional optimization (over all nonnegative functions) to a finite-dimensional one (over vectors of bin averages), with a quantified approximation error.

(ii) **Discretize** the finite-dimensional simplex to a lattice, with a quantified rounding error.

(iii) **Enumerate** every lattice point and verify that each one has autoconvolution peak exceeding the target threshold. If all are eliminated, the bound is proven.

Step (iii) is made tractable by pruning strategies (asymmetry, windowed bounds) and a multiscale cascade (coarse-to-fine refinement).

2 From Functions to Bin Averages

Our first order of business is to replace the infinite-dimensional optimization — over all nonnegative functions — with something finite-dimensional and computable.

2.1 The Partition

Fix a positive integer n (the "half-bin count"). Partition $(-\frac{1}{4}, \frac{1}{4})$ into $d = 2n$ equal bins:

$$I_j \;=\; \left(\frac{j}{4n}, \frac{j+1}{4n}\right), \qquad j = -n, -n+1, \ldots, n-1.$$

Each bin has width $\frac{1}{4n}$.

Remark 2.1. We use $d = 2n$ bins total, indexed from $-n$ to $n-1$. In the implementation, we re-index these as $0, 1, \ldots, d-1$ (i.e., $j \mapsto j+n$). The mathematics is the same either way.

2.2 Bin Averages

For any nonnegative f with $\operatorname{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ and $\int f = 1$, define the *bin averages*:

$$a_j \;=\; 4n \int_{I_j} f(x)\,dx \;\geq\; 0, \qquad j = 0, \ldots, d-1.$$

The factor $4n$ normalizes so that each a_j equals the average value of f on the bin I_j (since $|I_j| = \frac{1}{4n}$). The constraint $\int f = 1$ becomes

$$\sum_{j=0}^{d-1} a_j \;=\; 4n.$$

Definition 2.2. The *continuous simplex* is

$$A_n \;=\; \left\{ a \in \mathbb{R}^d_{\geq 0} \;\middle|\; \sum_{j=0}^{d-1} a_j = 4n \right\}.$$

Every nonnegative function f with $\int f = 1$ maps to a point in A_n, but not every point in A_n arises from a function (the map loses information about the shape of f within each bin).

2.3 The Lower Bound on $\|f * f\|_\infty$

Remarkably, the entire reduction rests on two elementary observations — one geometric, one analytic.

Lemma 2.3 (Reduction to Bin Averages — Lemma 1 of CS17)**.** *For any $f \in \mathcal{F}$ with $\int f = 1$ and corresponding bin averages $a = (a_0, \ldots, a_{d-1})$:*

$$\|f * f\|_{L^\infty} \;\geq\; \max_{\substack{2 \leq \ell \leq 2d \\ k}} \frac{1}{4n\ell} \sum_{\substack{0 \leq i,j \leq d-1 \\ k \leq i+j \leq k+\ell-2}} a_i\, a_j.$$

Proof sketch. **Ingredient 1: Support of cross-convolutions.** Let $f_i = f \cdot \mathbf{1}_{I_i}$ be the restriction of f to bin I_i. Then $f = \sum_i f_i$ and

$$(f * f)(t) \;=\; \sum_{i,j} (f_i * f_j)(t).$$

The convolution $f_i * f_j$ is supported on the Minkowski sum $I_i + I_j$, which is an interval of width $\frac{1}{2n}$. The key point: *the index $i + j$ determines which part of the convolution domain receives the contribution from $f_i * f_j$.*

Ingredient 2: Pigeonhole. For any nonnegative function h and interval J: $\|h\|_{L^\infty} \geq \frac{1}{|J|} \int_J h$.

Combining. Consider a "window" covering sum-indices $s \in \{k, k+1, \ldots, k+\ell-2\}$. The Minkowski sums for consecutive sum-indices overlap (their centers are spaced $\frac{1}{4n}$ apart, each of width $\frac{1}{2n}$), so their union forms an interval of length exactly $\frac{\ell}{4n}$ in the convolution domain. All cross-terms $f_i * f_j$ with $k \leq i + j \leq k + \ell - 2$ have support within this interval, contributing total L^1 mass $\sum \frac{a_i}{4n} \cdot \frac{a_j}{4n}$. Applying the pigeonhole bound:

$$\|f * f\|_{L^\infty} \;\geq\; \frac{1}{\ell/(4n)} \sum_{k \leq i+j \leq k+\ell-2} \frac{a_i\, a_j}{(4n)^2} \;=\; \frac{1}{4n\ell} \sum_{k \leq i+j \leq k+\ell-2} a_i\, a_j.$$

Taking the maximum over all windows gives the result. □

Remark 2.4 (Intuition)**.** Think of the bin averages a_i as describing "how much mass f puts in each bin." The products $a_i a_j$ measure how much the mass in bins i and j contribute to the autoconvolution. The window argument says: if too much product-mass falls in any fixed interval of the convolution domain, then $f * f$ must be large somewhere in that interval.

The reader may wonder: why not just take the narrowest possible window ($\ell = 2$) everywhere? Because narrow windows see fewer cross-terms, so their sums can be small even when mass is spread across many bins. The interplay between window width and the number of contributing pairs is what makes the optimization nontrivial.

3 Discretization of the Simplex

Having reduced the problem to minimizing over the continuous simplex A_n, we face a new obstacle: A_n is uncountable. To bring the problem within reach of a computer, we renormalize to the bin-mass coordinates

$$\mu_j \;=\; \frac{a_j}{4n}, \qquad \sum_{j=0}^{d-1} \mu_j = 1,$$

and approximate this normalized simplex by a finite lattice.

3.1 The Lattice

In these normalized coordinates, the natural grid has spacing $1/m$.

Definition 3.1. Fix a positive integer m (the "grid resolution"). The *discrete lattice* is

$$B_{n,m} \;=\; \left\{ b \in \left(\tfrac{1}{m}\mathbb{N}\right)^d \;\middle|\; \sum_{j=0}^{d-1} b_j = 1 \right\},$$

where each b_j is a nonnegative multiple of $\frac{1}{m}$.

Equivalently, writing $c_j = m \cdot b_j$, the lattice consists of nonnegative integer vectors $c = (c_0, \ldots, c_{d-1})$ with $\sum c_j = m$. These are the *compositions* of the integer m into d nonnegative parts.

Remark 3.2 (Implementation convention)**.** The implementation stores the integer composition c itself. The corresponding bin-average vector is

$$a_j \;=\; \frac{4n}{m}\, c_j.$$

Thus the discretization lemmas are most naturally phrased in the normalized mass coordinates $b_j = c_j/m$, while the test-value formulas remain most natural in the bin-average coordinates a_j.

The lattice is useful only if every continuous point has a nearby lattice point — otherwise gaps in the lattice could hide the minimizer.

Lemma 3.3 (Lattice as an Approximation — Lemma 2 of CS17)**.** *$B_{n,m}$ is a $\frac{1}{m}$-net of the normalized mass simplex. That is, for every $\mu \in \mathbb{R}^d_{\geq 0}$ with $\sum \mu_j = 1$, there exists $b \in B_{n,m}$ with $\|\mu - b\|_\infty \leq \frac{1}{m}$.*

Proof sketch. Given μ with $\sum \mu_j = 1$, round each coordinate to the nearest multiple of $\frac{1}{m}$, adjusting to maintain the sum constraint. Specifically, use a greedy rounding: process coordinates left to right, maintaining a running deficit $t_i = \sum_{j<i}(b_j - \mu_j)$. Round down first, then alternate to keep $-\frac{1}{m} < t_{i+1} \leq 0$. The last coordinate is set to enforce the exact sum. Each coordinate satisfies $|\mu_j - b_j| \leq \frac{1}{m}$. □

3.2 The Discretization Error Bound

Since we approximate the normalized mass vector μ by b with per-coordinate error at most $\frac{1}{m}$, the windowed test value changes by a bounded amount.

Lemma 3.4 (Discretization Error — corrected Lemma 3 of CS17)**.** *Let $a \in A_n$, write $\mu_i = a_i/(4n)$, and let $b \in B_{n,m}$ satisfy $\|\mu - b\|_\infty \le \frac{1}{m}$. Then, for any admissible window (ℓ, k),*

$$W_{\ell,k}(\mu) \;\ge\; W_{\ell,k}(b) \;-\; \frac{4n}{\ell}\left(\frac{2}{m} + \frac{1}{m^2}\right),$$

where

$$W_{\ell,k}(x) \;=\; \frac{4n}{\ell} \sum_{\substack{0 \le i,j \le d-1 \\ k \le i+j \le k+\ell-2}} x_i x_j.$$

Consequently, if $\tilde{b}_i = 4n\, b_i$ is the associated bin-average vector, then

$$\mathrm{tv}(a) \;\ge\; \mathrm{tv}(\tilde{b}) \;-\; 2n\left(\frac{2}{m} + \frac{1}{m^2}\right).$$

Proof sketch. This is the corrected CS17 discretization bound. For a fixed window, write $\mu = b + \varepsilon$. The perturbation contributes two error terms: a cross term of size $\frac{4n}{\ell} \cdot \frac{2}{m} W$, where $W \le 1$ is the total contributing mass in that window, and a quadratic term of size $\frac{4n}{\ell} \cdot \frac{1}{m^2}$. Summing them yields the per-window correction

$$\frac{4n}{\ell}\left(\frac{2}{m} + \frac{1}{m^2}\right).$$

Since $\ell \ge 2$, we have $\frac{4n}{\ell} \le 2n$, which gives the stated global bound for tv. □

Remark 3.5 (The correction term in practice)**.** The raw CS17 correction is $\frac{2}{m} + \frac{1}{m^2}$, but the global level-$n$ correction is $2n\left(\frac{2}{m} + \frac{1}{m^2}\right)$. At the starting level $n = 2$ and $m = 20$, this is 0.41. In the current codebase, this global quantity is used only for coarse caps and reporting; the actual pruning kernel uses the sharper window-dependent threshold described next.

3.3 The Refined (Dynamic) Correction

The global correction bound from Lemma 3.4 is pessimistic: it assumes the worst-case window length and replaces the actual contributing mass by its trivial upper bound. A tighter, position-dependent bound exploits the fact that narrow or edge-positioned windows involve fewer bins.

Lemma 3.6 (Dynamic Correction — Equation (1) of CS17)**.** *For a specific window at position k with length ℓ, let $W_{int} = \sum_{i \in \mathcal{B}(k,\ell)} c_i$ be the total integer mass in contributing bins, where $\mathcal{B}(k,\ell) = \{i : \exists\, j \text{ with } k \le i + j \le k + \ell - 2\}$. Then the* window-specific effective threshold *is:*

$$T_{\ell,k} \;=\; c_{\text{target}} + \frac{4n}{\ell} \cdot \frac{1 + 2\,W_{\text{int}}}{m^2}$$

rather than the coarse global threshold $c_{\text{target}} + 2n\left(\frac{2}{m} + \frac{1}{m^2}\right)$.

Proof. Let ws be the integer window sum of the autoconvolution of a composition c with $\sum_i c_i = m$. Since the implementation uses the rescaling $a_i = \frac{4n}{m} c_i$, the corresponding test value is

$$W_{\ell,k}(a) \;=\; \frac{4n}{m^2 \ell}\,\mathrm{ws}.$$

Therefore the corrected dynamic pruning condition is equivalent to

$$\mathrm{int_thresh} \;=\; \Big\lfloor \Big(c_{\mathrm{target}} \cdot m^2 \cdot \frac{\ell}{4n} + 1 + 2\,W_{\mathrm{int}} + 10^{-9} m^2 \Big)(1 - 4\epsilon_{\mathrm{mach}}) \Big\rfloor,$$

which is the corrected form now used in `run_cascade.py`. The additive $10^{-9}m^2$ term raises the threshold, while the factor $(1 - 4\epsilon_{\mathrm{mach}})$ lowers it slightly before flooring; the additive margin dominates, so the net effect is conservative. □

Remark 3.7 (Why dynamic is better)**.** The dynamic correction keeps track of the actual contributing mass W_{int} and retains the favorable factor $\frac{4n}{\ell}$. Long windows and edge windows therefore carry smaller corrections than the worst-case global bound, which is why the cascade can prune far more aggressively than a uniform-threshold search.

4 The Windowed Test Value

With the discretization machinery in hand, we can describe precisely what the algorithm computes at each lattice point.

4.1 Definition

Definition 4.1. For a mass vector $a = (a_0, \ldots, a_{d-1})$ with $a_i \geq 0$ and $\sum a_i = 4n$, the *autoconvolution* is the vector of length $2d - 1$:

$$\mathrm{conv}[s] \;=\; \sum_{\substack{i+j=s \\ 0 \leq i,j \leq d-1}} a_i\, a_j, \qquad s = 0, 1, \ldots, 2(d-1).$$

The *test value* is

$$\mathrm{tv}(a) \;=\; \max_{\substack{2 \leq \ell \leq 2d \\ 0 \leq k \leq 2(d-1)-(\ell-2)}} \frac{1}{4n\ell} \sum_{s=k}^{k+\ell-2} \mathrm{conv}[s].$$

In words: compute the autoconvolution, slide a window of each possible width ℓ across it, sum the $\ell - 1$ consecutive entries in each window position, normalize by $4n\ell$, and take the maximum.

Remark 4.2 (Integer implementation coordinates)**.** The code stores compositions $c = (c_0, \ldots, c_{d-1}) \in \mathbb{N}^d$ with $\sum_i c_i = m$ and recovers the bin-average vector by

$$a_i \;=\; \frac{4n}{m}\, c_i.$$

All pruning formulas below are stated in these implementation coordinates.

4.2 Efficient Computation via Prefix Sums

The test value can be computed efficiently using prefix sums. Let $P[s] = \sum_{k=0}^{s} \mathrm{conv}[k]$ be the prefix sum of the autoconvolution. Then the windowed sum from s_{lo} to s_{hi} is:

$$\sum_{s=s_{\mathrm{lo}}}^{s_{\mathrm{hi}}} \mathrm{conv}[s] \;=\; P[s_{\mathrm{hi}}] - P[s_{\mathrm{lo}} - 1] \qquad (\text{with } P[-1] = 0).$$

Remark 4.3 (Complexity)**.** Computing the autoconvolution takes $O(d^2)$ time. An FFT-based method would give $O(d \log d)$, but for the small d values used here the direct method is faster in practice. The prefix sum takes $O(d)$, and the complete window scan takes another $O(d^2)$. Thus the overall cost per lattice point is $O(d^2)$.

4.3 Reversal Symmetry

A key structural observation cuts the search space in half. If we reverse the order of the bins, the autoconvolution peak does not change — intuitively, because convolution does not care which direction we label the support.

Theorem 4.4 (Autoconvolution Reversal Symmetry)**.** *For any mass vector a,* $\mathrm{tv}(a) = \mathrm{tv}(\mathrm{rev}(a))$*, where* $\mathrm{rev}(a) = (a_{d-1}, \ldots, a_0)$*.*

Proof. The argument has three moving parts.

Step 1: Autoconvolution reversal. Let $a' = \mathrm{rev}(a)$, so $a'_i = a_{d-1-i}$. Then:

$$\mathrm{conv}'[s] \;=\; \sum_{i+j=s} a'_i a'_j \;=\; \sum_{i+j=s} a_{d-1-i}\, a_{d-1-j}.$$

Substituting $i' = d-1-i$, $j' = d-1-j$ (so $i' + j' = 2(d-1) - s$):

$$\mathrm{conv}'[s] \;=\; \mathrm{conv}[2(d-1) - s].$$

That is, the autoconvolution of the reversal is the reversal of the autoconvolution.

Step 2: Window-sum bijection. A window sum of width $w = \ell - 1$ starting at s_0 transforms under reversal to a window sum of the same width starting at $2(d-1) - s_0 - w + 1$:

$$\begin{aligned}\sum_{s=s_0}^{s_0+w-1} \mathrm{conv}'[s] &= \sum_{s=s_0}^{s_0+w-1} \mathrm{conv}[2(d-1) - s] \\ &= \sum_{s'=2(d-1)-s_0-w+1}^{2(d-1)-s_0} \mathrm{conv}[s'].\end{aligned}$$

This is a window of the same width w, so the multiset of all window sums at width w is identical for conv and conv$'$.

Step 3: Normalization preserves the maximum. The normalization $\frac{1}{4n\ell}$ depends only on ℓ, not the window position. Since the multiset of window sums is identical at each width, $\mathrm{tv}(a) = \mathrm{tv}(a')$. □

Corollary 4.5 (Canonical Filter)**.** *It suffices to check only* canonical *integer compositions c with $c \le \mathrm{rev}(c)$ lexicographically. The associated bin-average vectors have the same test value, so this halves the search space (up to palindromes).*

Even after exploiting reversal symmetry, the number of implementation states to check — roughly $\binom{m+d-1}{d-1}/2$ — grows rapidly with d and m. To keep the computation tractable, we need ways to rule out large swathes of the simplex without computing the full test value.

5 Pruning Strategies

5.1 Asymmetry Pruning

At first glance, the following bound might seem too crude to be useful: it ignores everything about f except how much mass sits on each half of the support. Yet in practice, this single test eliminates the vast majority of configurations at every cascade level.

Theorem 5.1 (Asymmetry Bound)**.** *Let $f \ge 0$ with $\mathrm{supp}(f) \subseteq (-\frac{1}{4}, \frac{1}{4})$ and $\int f = 1$. Let $L = \int_{-1/4}^{0} f$ be the left-half mass. Then:*

$$\|f * f\|_{L^\infty} \;\ge\; 2L^2.$$

Proof. Consider the left-left convolution:

$$(f * f)(t) \;=\; \int f(x)\, f(t-x)\, dx \;\ge\; \int_{-1/4}^{0} f(x)\, f(t-x)\, \mathbf{1}_{[-1/4,0]}(t-x)\, dx$$

for $t \in (-\frac{1}{2}, 0)$. By Fubini's theorem, the total mass of $f * f$ restricted to $(-\frac{1}{2}, 0)$ is at least L^2 (the left-left contribution). Since $|(-\frac{1}{2}, 0)| = \frac{1}{2}$:

$$\|f * f\|_{L^\infty} \;\ge\; \frac{L^2}{1/2} \;=\; 2L^2.$$

By symmetry, the same holds for the right-half mass $R = 1 - L$: $\|f * f\|_\infty \ge 2R^2$. Therefore $\|f * f\|_\infty \ge 2\max(L^2, R^2)$. □

Corollary 5.2 (Asymmetry Threshold)**.** *If $2L^2 \ge c_{\mathrm{target}}$, i.e., $L \ge \sqrt{c_{\mathrm{target}}/2}$, then the configuration is ruled out. The threshold left-mass fraction is:*

$$p^* \;=\; \sqrt{c_{\mathrm{target}}/2}.$$

For $c_{\mathrm{target}} = 1.4$: $p^* \approx 0.8367$.

Remark 5.3 (Exactness for step functions and under refinement)**.** For a step function already represented by a composition $c = (c_0, \ldots, c_{d-1})$ with $\sum c_i = m$, the left-half mass is

$$L \;=\; \sum_{i<n} \frac{c_i}{m},$$

because the midpoint $x = 0$ lies exactly on a bin boundary. This quantity is also preserved exactly under dyadic refinement: the left child bins sum to the left parent bins. Thus asymmetry pruning is an exact statement for discrete step functions inside the cascade.

Remark 5.4 (Continuous-transfer caveat)**.** The asymmetry bound is a *direct* L^∞ bound on $f * f$. It does *not* go through the discretization framework of Lemma 3.4. Accordingly, one must distinguish two uses of asymmetry:

- for an *exact step function* represented by a composition c, the criterion is immediate and needs no correction term;
- for the *continuous-to-discrete transfer*, one must analyze how the chosen discretization records left- and right-half masses.

In the cumulative-floor discretization formalized in Lean, the discrete left mass records $\lfloor mL \rfloor / m$, so a left-heavy certificate transfers directly to the underlying continuous profile, whereas the symmetric right-heavy condition needs an additional lemma or a safety margin. The final theorem below is therefore phrased conditionally on whatever asymmetry-transfer principle one adopts.

Remark 5.5 (What the current implementation actually prunes by)**.** Earlier drafts experimented with additional block-sum and two-peak heuristics. The current CPU cascade does not rely on them. The present code prunes by asymmetry, the corrected window-dependent dynamic threshold, reversal canonicalization, and the single-bin cap used during child generation. In the proof narrative below, the window certificates and the single-bin bound are the unconditional continuous certificates; asymmetry is best viewed as an exact discrete pruning rule together with a separate transfer question.

5.2 Canonical Symmetry Exploitation

If c is an implementation-space composition and $\tilde{c}_i = \frac{4n}{m} c_i$ is its associated bin-average vector, then Theorem 4.4 gives $\mathrm{tv}(\tilde{c}) = \mathrm{tv}(\widetilde{\mathrm{rev}(c)})$. This motivates the following notion of canonicality.

Definition 5.6. A composition c is *canonical* if $c \leq \mathrm{rev}(c)$ lexicographically. That is, comparing c_0 with c_{d-1}, then c_1 with c_{d-2}, etc., c comes first (or they are equal, i.e., c is a palindrome).

Proposition 5.7 (Canonical Count)**.** *For integer compositions of a fixed total mass M into d parts, the number of canonical compositions is $\frac{N+P}{2}$, where $N = \binom{M+d-1}{d-1}$ is the total count and P is the number of palindromes.*

Proof. Every composition falls into exactly one of three categories: palindromes (those equal to their own reversal), strictly canonical non-palindromes ($b < \mathrm{rev}(b)$), and their reversals. Since reversal is an involution with no fixed points outside the palindromes, the second and third categories have equal size C. Counting gives $N = P + 2C$, so the canonical count is $P + C = (N + P)/2$. □

Remark 5.8 (Loop-bound tightening for $d = 4$)**.** For $b = (c_0, c_1, c_2, c_3)$ with total mass M and $c_3 = M - c_0 - c_1 - c_2$, canonicality ($b \leq \mathrm{rev}(b)$) requires $c_0 \leq c_3$, giving three loop bounds:

- $c_0 \leq \lfloor M/2 \rfloor$ (if $c_0 > M/2$ then $c_3 < c_0$).
- $c_1 \leq M - 2c_0$ (need $c_2 \geq 0$ with $c_1 + c_2 \leq M - 2c_0$).

- $c_2 \le M - 2c_0 - c_1$ (ensures $c_3 \ge c_0$).

Plus a palindrome filter: when $c_0 = c_3$, require $c_1 \le c_2$.

For $d = 6$, analogous bounds apply: $c_0 \le \lfloor M/2 \rfloor$, $c_4 \le M - c_0 - c_1 - c_2 - c_3 - c_0$, with equality checks for the inner pairs when outer pairs tie.

6 The Multiscale Cascade

Brute-force enumeration at fine resolution is hopelessly infeasible — even modest parameters yield billions of lattice points. The key idea, borrowed from multigrid methods in numerical analysis, is to start coarse: most configurations can be ruled out cheaply at low resolution, and only the stubborn survivors need to be examined more closely.

6.1 Overview

Step 1: Start coarse. At level L_0, enumerate all canonical compositions of m into $d_0 = 2n_0$ parts (e.g., $n_0 = 2$, $d_0 = 4$).

Step 2: Prune. For each composition, apply asymmetry pruning and then scan windows using the corrected dynamic threshold. If some window certifies the target bound, the composition is eliminated.

Step 3: Refine survivors. Each surviving composition at resolution d is refined to resolution $2d$ by *dyadic splitting*: each bin is split into two sub-bins.

Step 4: Repeat. Prune the refined compositions. Continue until either all are eliminated (proof complete) or a computational budget is reached.

6.2 Dyadic Splitting (Refinement)

Given a parent composition $b = (b_0, \ldots, b_{d-1})$ at resolution d, the refinement produces children $c = (c_0, c_1, \ldots, c_{2d-1})$ at resolution $2d$, where:

$$c_{2i} + c_{2i+1} \;=\; b_i \qquad \text{for each } i = 0, \ldots, d-1,$$

with $c_{2i}, c_{2i+1} \ge 0$ and both being nonnegative integers.

6.3 Energy Cap

One might worry that generating all possible children is wasteful. Indeed, many children are immediately ruled out by the single-bin diagonal window. The energy cap avoids generating them in the first place.

Recall that $a_j = \frac{4n_{\text{child}}}{m} c_j$ and the $\ell = 2$ diagonal window at position $2j$ gives

$$\text{tv}(a) \;\ge\; d_{\text{child}} \frac{c_j^2}{m^2}, \qquad d_{\text{child}} = 2n_{\text{child}}.$$

This yields the test-value cap

$$x_{\text{cap}}^{\text{TV}} \;=\; \left\lfloor m \sqrt{\frac{c_{\text{target}} + 2n_{\text{child}}\left(\frac{2}{m} + \frac{1}{m^2}\right) + 10^{-9}}{d_{\text{child}}}} \right\rfloor.$$

The implementation also applies the sharper direct cap

$$x_{\text{cap}}^{\text{CS}} \;=\; \left\lfloor m\sqrt{\frac{c_{\text{target}}}{d_{\text{child}}}} \right\rfloor,$$

and uses $x_{\text{cap}} = \min\bigl(x_{\text{cap}}^{\text{TV}}, x_{\text{cap}}^{\text{CS}}\bigr)$.

Proposition 6.1 (Energy Cap Soundness)**.** *If a child has some bin $c_j > x_{\text{cap}}^{\text{CS}}$, then any nonnegative function with those child-bin masses already satisfies $\|f * f\|_{L^\infty} > c_{\text{target}}$.*

Proof. Let $M_j = c_j/m$ be the mass in the jth child bin. Restricting f to that single bin produces a nonnegative function f_j supported on an interval of length $1/d_{\text{child}}$ and with total mass M_j. By the same averaging argument used for asymmetry pruning,

$$\|f * f\|_{L^\infty} \;\geq\; \|f_j * f_j\|_{L^\infty} \;\geq\; \frac{M_j^2}{1/d_{\text{child}}} \;=\; d_{\text{child}}\,\frac{c_j^2}{m^2}.$$

If $c_j > m\sqrt{c_{\text{target}}/d_{\text{child}}}$, the right-hand side exceeds c_{target}, which proves the claim. □

Remark 6.2 (Tighter Cauchy–Schwarz cap)**.** The looser cap $x_{\text{cap}}^{\text{TV}}$ mirrors the older global correction narrative, while the direct cap $x_{\text{cap}}^{\text{CS}}$ is the effective one in the current implementation. It is tighter and does not need any discretization correction term.

6.4 Cascade Coverage

Theorem 6.3 (Terminal-Grid Coverage)**.** *If the cascade terminates with zero survivors at some level L_k (with resolution $d_k = d_0 \cdot 2^k$), then every composition at that terminal resolution has some ancestor on its refinement path that is covered by at least one valid certificate: asymmetry pruning, the single-bin cap, or a specific window certificate satisfying the corrected dynamic threshold of Lemma 3.6.*

Proof outline. The content of the cascade is not merely that "no survivor remains"; it is that every terminal discrete configuration has acquired a certificate. Level L_0 is complete up to reversal symmetry, and the refinement step generates every child of every surviving parent except those excluded by the single-bin cap, which are already covered by the direct bound from Section 6.3. Canonicalize-after- test handling ensures that reversal symmetry does not lose any child- level configuration. Thus each prune event cuts off an entire subtree of descendants. Therefore, if the survivor set is empty at level L_k, every terminal composition has some ancestor that was eliminated either by asymmetry, by the single-bin cap, or by a concrete window (ℓ, s_{lo}) whose threshold test passes. □

6.5 Canonical Handling Across Levels

At level L_0, only canonical compositions are generated. At levels L_1 and beyond, the situation is more subtle:

- Children of a canonical parent P are *not* necessarily canonical.
- The reversal $\text{rev}(P)$ is *not* in the parent list (unless P is a palindrome), so its children are not generated directly.

- However, if C is a child of P, then $\mathrm{rev}(C)$ is a child of $\mathrm{rev}(P)$. Since $\mathrm{tv}(C) = \mathrm{tv}(\mathrm{rev}(C))$ (Theorem 4.4), testing C suffices for both.

Therefore at levels ≥ 1:

1. Generate *all* children of each parent (no canonical filter during generation).
2. Test each child against the dynamic threshold.
3. *Canonicalize* survivors: replace each with $\min(C, \mathrm{rev}(C))$ lexicographically.
4. *Deduplicate*: remove duplicate canonical forms (children from different parents, or a palindrome parent producing both C and $\mathrm{rev}(C)$).

Proposition 6.4 (Completeness of Canonical Handling)**.** *This procedure tests every composition at the child resolution: if C' is any child-level composition and $P' = parent(C')$, then either P' or $\mathrm{rev}(P')$ is in the parent list, and C' or $\mathrm{rev}(C')$ is among the tested children.*

6.6 Fused Generate-and-Prune Kernel

At deep cascade levels, each parent may have millions of children. The fused kernel generates and tests each child inline via a Cartesian-product odometer, never materializing the full children array:

- **Incremental autoconvolution:** when only the last cursor position changes, the raw autoconvolution is updated in $O(d)$ instead of $O(d^2)$.
- **Quick-check:** re-tries the previous killing window before a full scan.
- **Subtree pruning:** on deep carries, checks the partial autoconvolution of fixed bins against worst-case unfixed mass; if it exceeds the threshold, the entire subtree is skipped.

7 The Complete Proof

7.1 Algorithm

Algorithm 1 Cascade Search For Terminal Coverage

Require: c_{target}, grid resolution m, starting half-bins n_0, max levels $L_{\max}$

```
d_0 ← 2n_0
Generate all canonical compositions of m into d_0 parts ▷ implementation coordinates
Apply asymmetry pruning (threshold √(c_target/2))
For every remaining composition, scan all windows and prune if some window exceeds
  the dynamic threshold from Lemma 3.6
survivors_0 ← unpruned compositions
for k = 1, 2, ..., L_max do
    d_k ← 2d_{k−1};   n_k ← 2n_{k−1}
    Compute the single-bin child cap x_cap = min(x_cap^TV, x_cap^CS)
    for all parent P ∈ survivors_{k−1} do
        for all children C of P (dyadic split with energy cap) do
            Apply asymmetry pruning to C
            if not asymmetry-pruned then
                Scan all windows of C via prefix-sum autoconvolution
                if some window exceeds the dynamic threshold then
                    Prune C
                else
                    C survives; canonicalize and store
                end if
            end if
        end for
    end for
    Deduplicate survivors (canonicalize → sort → scan)
    survivors_k ← deduplicated survivors
    if |survivors_k| = 0 then
        return ZERO-SURVIVORS
    end if
end for
return INCONCLUSIVE: |survivors_{L_max}| compositions remain
```

7.2 Main Theorem

Theorem 7.1 (Conditional Main Result)**.** *If Algorithm 1 terminates with the status* ZERO-SURVIVORS*, and if every composition eliminated solely by asymmetry pruning is equipped with a valid continuous transfer argument at the relevant level, then* $C_{1a} \geq c_{\text{target}}$.

Proof. Combine three ingredients:

1. the reduction from continuous functions to windowed test values (Lemma 2.3);
2. the corrected discretization and dynamic-threshold bounds (Lemmas 3.4 and 3.6); and

3. the cascade completeness statement summarized in Theorem 6.3.

If Algorithm 1 returns ZERO-SURVIVORS, then some level has zero survivors, so every configuration at that terminal resolution has some ancestor carrying one of the certificates listed in Theorem 6.3. The single-bin cap is a direct L^∞ lower bound, and the window certificates combine with Lemmas 3.4 and 3.6 to transfer the bound to the continuous problem. By the additional assumption in the statement, the remaining asymmetry eliminations also transfer. Therefore $C_{1a} \geq c_{\text{target}}$. □

8 Correctness Invariants

Stating the algorithm is only half the battle. For the proof to carry mathematical weight, we must verify that the implementation faithfully realizes the specification. Four invariants pin down exactly what "faithful" means.

8.1 Invariant 1: Complete Enumeration

Every implementation-space composition $c \in \mathbb{N}^d$ with $\sum_i c_i = m$ must be either directly checked or shown to be covered by a checked ancestor and all its refinements.

- At L_0: all $\binom{m+d_0-1}{d_0-1}$ compositions are enumerated (in canonical form, with reversal symmetry). Verified by comparing the generator output count against the stars-and-bars formula, and by checking completeness, uniqueness, non-negativity, and sum constraints.
- At L_k ($k \geq 1$): all children of all surviving parents are generated (no sampling or probabilistic skipping).

8.2 Invariant 2: All Windows Checked

For each composition that is not eliminated by asymmetry, every valid window (ℓ, k) with $2 \leq \ell \leq 2d$ and all valid starting positions k must be tested before the composition is declared a survivor. Early exit occurs only *after* a pruning window is found — never for survivors.

8.3 Invariant 3: Conservative Arithmetic

Floating-point arithmetic is the Achilles' heel of computer-assisted proofs: a single rounding error in the wrong direction could cause a valid configuration to be falsely pruned, invalidating the entire argument. We guard against this as follows:

- Integer coordinates: the autoconvolution is computed in integer arithmetic ($c_i c_j$ are exact integers), avoiding round-off entirely.
- Conservative threshold: the code evaluates the corrected integer threshold

$$c_{\text{target}} m^2 \frac{\ell}{4n} + 1 + 2W_{\text{int}},$$

  together with an added $10^{-9}m^2$ margin and a final $(1-4\epsilon_{\text{mach}})$ factor before flooring.

- The multiplicative factor lowers the threshold slightly, but the additive $10^{-9}m^2$ margin is overwhelmingly larger, so the net effect remains conservative: the code may miss a borderline prune, but it does not falsely prune.

8.4 Invariant 4: Refinement Completeness

Every refinement of a surviving parent must be checked at the finer level. The dyadic splitting with energy cap generates all valid children (Section 6.3), and children exceeding the energy cap are provably pruned.

9 Computational Verification

The correctness of the implementation has been verified through multiple independent checks.

9.1 Composition Generators

- **Completeness**: For 7 parameter combinations (d, S), the generator output count matches $\binom{S+d-1}{d-1}$ exactly. All outputs sum to S, are non-negative, and are unique.
- **Specialized $\equiv$ generic**: The $d = 4$ and $d = 6$ specialized generators produce identical sets to the generic stack-based generator.
- **Canonical correctness**: For 8 parameter combinations, canonical generators produce exactly $\frac{N+P}{2}$ outputs, all satisfying $b \leq \mathrm{rev}(b)$, with every non-canonical b having $\mathrm{rev}(b)$ in the output.
- **Batch boundaries**: Across 9 configurations and 8 batch sizes, output is identical regardless of where batch boundaries fall (160 checks, all pass).

9.2 Test Value Computation

- Reversal symmetry ($\mathrm{tv}(b) = \mathrm{tv}(\mathrm{rev}(b))$) verified exhaustively for $d = 4, S = 8$ (165 compositions) and $d = 6, S = 6$ (462 compositions), plus 1,500 random tests. Zero mismatches.
- Pure-Python reference implementation agrees with Numba JIT kernel on all 891 canonical compositions at $d = 4, m = 20$.

9.3 Pruning Correctness

- Window pruning: zero compositions are pruned whose true test value is below the dynamic threshold.
- Window pruning: zero compositions fail to be pruned when their true test value exceeds the dynamic threshold.
- Asymmetry filter: exhaustive step-function checks at coarse levels confirm the expected lower bound on the exact discretized profiles, and the filter is exactly symmetric under reversal.

9.4 MATLAB Baseline Comparison

The cascade prover's dynamic threshold formula has been verified algebraically equivalent to the original MATLAB code of Cloninger–Steinerberger, with the Python implementation being strictly more conservative (harder to prune) due to the explicit $10^{-9}m^2$ safety margin.

9.5 Lean Formalization Status

This repository also contains a consolidated Lean 4 development in `lean/complete_proof.lean`. It formalizes a substantial portion of the discrete infrastructure used by the cascade: foundational discretization lemmas, reversal symmetry, composition counting, refinement mass preservation, the basic asymmetry bound, and several kernel invariants related to W_{int}, pruning monotonicity, and integer arithmetic.

At the same time, the Lean file is not yet a completely end-to-end formalization of the final theorem. In particular, the corrected per-window discretization bound enters there as an assumed lemma, the fully symmetric no-margin asymmetry transfer is not yet formalized, and the final cascade-to-continuous reduction remains only partially formalized. The TeX proof and the Lean development should therefore be viewed as complementary rather than identical artifacts.

10 Discussion

10.1 Relationship to the Original CS17 Proof

Our framework is a direct extension of Cloninger & Steinerberger (2017). The key differences:

1. **Dynamic correction**: We use a sharper per-window correction rather than only a coarse global bound. This improves pruning while preserving rigor.

2. **Multiscale cascade**: CS17 used $m = 50$ and refined from $n = 3$ through $n = 24$. We use $m = 20$ with dyadic refinement from $d = 4$ through $d = 64$, exploiting the fused kernel's efficiency.

3. **Fused generation**: Instead of materializing all children and then testing, we generate and test each child inline, reducing memory from $O(N_{\text{children}} \cdot d)$ to $O(d)$ per parent.

4. **Integer arithmetic**: Autoconvolution and threshold comparisons use exact integer operations, with floating-point involved only in the conservative threshold computation.

10.2 Limitations

- The proof is *computer-assisted*: the mathematical argument reduces correctness to a finite computation, but that computation must be performed by a machine. The invariants in Section 8 ensure that the computation is trustworthy.

- The approach is limited by the exponential growth of the composition count with d and m. The multiscale cascade mitigates this, but very aggressive targets (c_{target} close to C_{1a}) will require more cascade levels and higher computational cost.
- As written, the end-to-end theorem isolates one remaining transfer issue: the current implementation's symmetric no-margin asymmetry rule is exact for step functions, but an explicit continuous transfer lemma is still needed if one wants that rule itself to serve as an unconditional terminal certificate.

Acknowledgments

This work builds on the foundational algorithm of Cloninger and Steinerberger [1]. The constant C_{1a} is tracked in Tao et al.'s optimization constants repository [5].

References

[1] A. Cloninger and S. Steinerberger, *On Suprema of Autoconvolutions with an Application to Sidon Sets*, Proc. AMS **145**(8), 3191–3200, 2017. arXiv:1403.7988.

[2] M. Matolcsi and C. Vinuesa, *Improved bounds on the supremum of autoconvolutions*, J. Math. Anal. Appl. **372**(2), 439–447, 2010. arXiv:0907.1379.

[3] E. White, *New upper bounds on the supremum of autoconvolutions*, arXiv:2210.16437, 2022.

[4] E. Boyer and Z. Li, *Improved lower bounds on autoconvolution suprema*, 2025. arXiv:2506.16750.

[5] T. Tao et al., *Optimization Problems Repository*, https://teorth.github.io/optimizationproblems/constants/1a.html.